

Frontend Consistency Plan — Landing como Referência

1. Por que este documento existe
2. A landing é boa referência? Sim — por quê
3. Estado atual do resto do app (gaps vs. landing)
4. O que manter do sistema atual (não é tudo descartável)
5. Plano de implementação
6. Riscos / o que não fazer
7. Resumo executivo

1. Por que este documento existe

Hoje o frontend do XiaoLee tem **dois sistemas de design coexistindo** sem que isso tenha sido uma decisão explícita:

- src/components/landing/*** — Hero, Sections, primitives, ícones e strings da landing page (**/landing**).
- O resto do app — **/dashboard** , **/campaigns** , **/notifications** , **/onboarding** , **/traction** e a **Navbar** .

Cruzando datas de commit, a ordem real dos fatos foi:

Data	Evento
2026-05-09	docs/DESIGN_SYSTEM.md v3 documenta o padrão pink/fuchsia "clean", SVG inline por arquivo, max-w-2xl single-column como "mobile-first"
2026-05-10	dashboard/page.tsx (e as outras páginas do app) construídas seguindo esse padrão
2026-05-31	Landing page (Hero.tsx , Sections.tsx , primitives.tsx , icons.tsx , strings.ts) criada — um salto de qualidade na componentização e na responsividade

Conclusão prática: a landing não é um experimento paralelo, é a versão mais recente do design do produto. Ela simplesmente nunca foi retroportada para o resto do app nem para o **DESIGN_SYSTEM.md** , que ainda descreve o padrão de maio como se fosse o atual. Esse documento substitui esse vácuo: define a landing como fonte da verdade e traça o caminho para alinhar o resto do app a ela.

2. A landing é boa referência? Sim — por quê

Internamente, a landing é o único canto do frontend com disciplina de componentização real:

- Primitives reutilizadas em todo lugar:** **Reveal** , **SectionHead** , **EyeBrow** , **CountUp** , **XiaoleeBubble** (**primitives.tsx**) são importadas por **todas** as 7 seções (**Pillars** , **SayItGrid** , **HowItWorks** , **Campaigns** , **Token** , **Channels** , **FinalCTA** , **Footer**). Nenhuma seção reimplementa um header de seção do zero.
- Ícones centralizados:** um único **icons.tsx** , zero duplicação.
- Strings isoladas:** **strings.ts** com **useLandingT** , **ROTATE** , **APP_URL** — i18n nunca hardcoded.
- Responsividade de verdade:** 36 ocorrências de **sm:/md:/lg:** só em **Hero.tsx** + **Sections.tsx** — grid muda de coluna, nav vira **hidden md:flex** , tipografia usa **clamp()** fluido. É mobile-first no sentido correto: a base é mobile, e a UI **escala** para telas maiores.
- Tom visual consistente e nomeado:** **font-display** , **text-ink** , **text-grad** , **.glass** , **.ring-soft** , animações **.reveal** — vocabulário visual com nome, não classes Tailwind soltas reinventadas a cada arquivo.

- **Paleta compatível com o resto do app:** continua fúcsia/purple/pink, então adotar a landing como referência **não é um rebrand** — é elevar o nível de execução de uma paleta que já é a mesma.

O único problema da landing é que esse vocabulário (`text-grad` , `.glass` , `.reveal` etc.) está fisicamente preso ao seletor `.xl-landing` em `landing.css` , então hoje é **impossível** usá-lo fora de `/landing` mesmo se alguém quisesse.

3. Estado atual do resto do app (gaps vs. landing)

Página/arquivo	Breakpoints (sm/md/lg)	Ícones	i18n	Componentização
<code>landing/*</code> (referência)	36+	centralizados (<code>icons.tsx</code>)	<code>strings.ts</code>	<code>primitives.tsx</code> reusado
<code>dashboard/page.tsx</code>	0	12 SVGs inline próprios	<code>t()</code> ok	<code>StatCard</code> -like reescrito local
<code>notifications/page.tsx</code>	0	emojis + gradientes inline	<code>t()</code> ok	stats reescritos local
<code>onboarding/page.tsx</code>	0	5 SVGs inline próprios	hardcoded EN	form custom, sem reuso
<code>traction/page.tsx</code>	0	7 SVGs inline próprios	hardcoded EN	<code>StatCard</code> local (3ª implementação)
<code>CampanhasNew.tsx</code>	0	—	—	—
<code>dashboard/TokenomicsCard.tsx</code>	0	+1 ícone duplicado	<code>t()</code> ok	conteúdo desatualizado (Solana/SPL vs. arquitetura Arc atual)
<code>Navbar.tsx</code>	usa sm/md/lg corretamente	<code>@heroicons/react</code> (4ª fonte de ícone)	<code>t()</code> parcial	monólito de 470 linhas, <code>console.log</code> de debug

Achados centrais:

1. **Zero breakpoints fora da landing e da Navbar** — as páginas do app ficam em coluna estreita (`max-w-2xl`) mesmo em desktop. Isso era uma escolha documentada no `DESIGN_SYSTEM.md` v3 (seção 1, item 6), não um acidente — mas é a escolha que a landing já superou.
2. **5 fontes de ícones diferentes** (`landing/icons.tsx` , SVGs inline em 4 arquivos distintos do app, `@heroicons/react` na Navbar) com sobreposição (`IconCheck` , `IconActivity` , `IconUsers` , `IconDollar` / `IconZap`) redeclarados quase identicamente em 3+ lugares).
3. **i18n incompleto** — `onboarding` e `traction` não passaram pelo Context de idioma.
4. **Vocabulário de tom indisponível fora da landing** — `text-grad` , `.glass` , `.reveal` , `font-display` só existem sob `.xl-landing` .

5. **Conteúdo desatualizado** — `TokenomicsCard` ainda fala de Solana/SPL Token-2022; a arquitetura atual (ver `CLAUDE.md`) pivotou para Arc/Circle/USDC.

4. O que manter do sistema atual (não é tudo descartável)

Nem tudo em `DESIGN_SYSTEM.md` v3 está errado — algumas regras são boas e devem sobreviver à migração:

- Regra de contraste (seção 9): proibir `text-gray-400/70` etc. e exigir `text-gray-400` sólido — continua válida, inclusive a landing já segue isso.
- Estrutura de i18n via Context + `locales/*.json` — mantém, só precisa de cobertura completa.
- Paleta semântica (emerald=sucesso, amber=pendente, red=erro) — mantém, é compatível com a landing.
- "Sem emojis de UI, emojis só na personalidade da Xiaolee" — mantém, e a landing já segue essa regra (emojis só nas falas do chat simulado).

O que muda é a **camada estrutural**: breakpoints, primitivas compartilhadas, ícones centralizados, tokens de tom — isso vem da landing.

5. Plano de implementação

Fase 1 — Destravar o vocabulário visual (pré-requisito de tudo)

- Mover os tokens de `landing.css` (`font-display`, `text-ink`, `text-grad`, `.glass`, `.ring-soft`, `@keyframes` de reveal/floaty) de dentro do escopo `.xl-landing` para `globals.css`, disponíveis no app inteiro.
- Validar que isso não quebra o visual atual da landing (ela deve continuar idêntica).
- **Esforço**: baixo · **Bloqueia**: Fases 2–4

Fase 2 — Unificar ícones

- Promover `landing/icons.tsx` para `src/components/icons.tsx` (escopo global), migrando para lá os ícones reais e únicos hoje espalhados em `dashboard/page.tsx`, `onboarding/page.tsx`, `traction/page.tsx`, `TokenomicsCard.tsx`.
- Deletar duplicatas; manter `@heroicons/react` só onde já está bem usado (Navbar) **ou** migrar a Navbar também, se o time topa.
- **Esforço**: baixo-médio · **Impacto**: alto (maior fonte de inconsistência hoje)

Fase 3 — Extrair primitivas compartilhadas

- Promover `Reveal`, `SectionHead` / `EyeBrow` -equivalente para `src/components/ui/`.
- Criar um `StatCard` único reaproveitável (hoje reimplementado em `dashboard`, `traction`, `notifications`) baseado no padrão de card da landing (`rounded-2xl`, `glass` opcional).
- **Esforço**: médio · **Depende de**: Fase 1 e 2

Fase 4 — Responsividade real nas páginas do app

- Reescrever os grids fixos (`grid-cols-2` , `grid-cols-3`) das páginas de app com breakpoints (`sm` : / `md` : / `lg` :), seguindo o padrão de `Sections.tsx` (ex.: stats em coluna única no mobile, 2-3-4 colunas conforme a tela cresce).
- Trocar o teto de `max-w-2xl` fixo por containers que escalam em telas maiores (`max-w-2xl lg:max-w-4xl` , como referência — ajustar por página), espelhando `max-w-[1180px]` da landing nas páginas que fazem sentido ter mais respiro.
- **Esforço:** médio-alto · **Depende de:** Fase 3 (para já consumir os componentes novos em vez de retrabalhar 2x)

Fase 5 — Acabamentos

- Completar i18n em `onboarding` e `traction` .
- Quebrar `Navbar.tsx` em subcomponentes (`NavLinks` , `UserDropdown` , `LangToggle` já existe duplicado — landing tem o seu próprio `LangToggle` , vale unificar) e remover `console.log` de debug.
- Atualizar `TokenomicsCard` para refletir a arquitetura Arc/Circle atual (alinhar com `CLAUDE.md` e `docs/workflows/ARC_LEPTON_ARCHITECTURE.md`).
- **Esforço:** baixo-médio

Fase 6 — Documentação

- Reescrever `docs/DESIGN_SYSTEM.md` (ou criar v4) para descrever o sistema pós-migração com a landing como fonte da verdade, substituindo a seção 1 item 6 ("mobile-first = `max-w-2xl` fixo") pela definição correta usada na landing (mobile-first com breakpoints escaláveis).
- **Esforço:** baixo · **Sem isso, a próxima página nova volta a divergir.**

6. Riscos / o que não fazer

- **Não** tratar isso como rebrand — a paleta já é compatível, então não há motivo para mexer em cores, só em estrutura/tokens/responsividade.
- **Não** migrar tudo de uma vez. A ordem das fases importa porque 3 e 4 dependem de 1 e 2 — pular a ordem gera retrabalho (construir componente novo em cima de classe que ainda não existe fora da landing).
- **Não** prometer prazo amarrado ao deadline do Lepton (6 jul) sem antes confirmar com o time se vale a pena gastar ciclo de frontend nisso versus os P0 do hackathon (`arc_x402` , tx USDC reais) — este plano é sobre consistência de produto, não é bloqueador de submissão.

7. Resumo executivo

A landing (31/05) é a versão mais madura do design do XiaoLee e deve virar a referência única. O trabalho não é "escolher entre dois estilos" — é propagar uma evolução que já existe para o resto do app, que ficou preso ao padrão de antes (maio). O maior ganho de esforço/impacto é a Fase 1+2 (destravar tokens + unificar ícones): baixo custo, desbloqueia tudo, e já elimina a maior fonte visível de inconsistência hoje.